

## 论文解构报告

# DynaPipe: Dynamic Layer Redistribution for Efficient Serving of LLMs with Pipeline Parallelism

Hongxin Xu, Tianyu Guo, Xianwei Zhang

流水线并行

LLM 推理

负载均衡

KV 缓存迁移

延迟预测

源文件：本地上传文件（无外部链接）

代码：<https://github.com/xhx1022/DynaPipe>

生成时间：2026-07-26

# 目录

---

- 摘要翻译 ..... 3
  - 1. 一句话总览 ..... 3
  - 2. 阅读范围与证据状态 ..... 3
  - 3. 根问题：论文究竟要解决什么 ..... 3
  - 4. 旧方法为何不够 ..... 4
  - 5. 核心洞见与假设 ..... 4
  - 6. 方法：从洞见到可执行系统 ..... 5
  - 7. 为什么它可能有效 ..... 5
  - 8. 实验与指标：证据是否支撑主张 ..... 6
  - 9. 图表解读与论证关系 ..... 7
  - 10. 完整因果推理树 ..... 9
  - 11. 结论、贡献与边界 ..... 9
  - 12. 术语与第一性原理学习路径 ..... 10

# 摘要翻译

为加速大语言模型（LLM）推理，流水线并行将模型层划分为多个顺序阶段，每个阶段分配给不同设备并发执行。然而，该方法常因尾部阶段计算不均衡而产生流水线气泡：上游阶段仅执行层前向计算，而最后阶段还需处理采样等额外后处理任务，从而引入显著延迟。这种工作负载差异导致流水线错位，使上游阶段被迫空闲，整体性能下降。现有框架通常在各阶段间均匀分配层，未考虑计算负载差异。为此，我们提出 DynaPipe，一种动态层重分配方案，通过实时预测执行延迟来自适应地平衡计算负载。此外，我们引入了一种异步键值（KV）缓存迁移协调机制，以在推理过程中实现非阻塞的层重分配。在代表性大语言模型上的实验表明，DynaPipe 在多种工作负载下将平均端到端请求延迟降低了 8% 至 41%，优于现有最先进的流水线并行系统。我们的实现已在 <https://github.com/xhx1022/DynaPipe> 公开。

## 1. 一句话总览

### 论文元数据

| 字段        | 内容                                                                                             |
|-----------|------------------------------------------------------------------------------------------------|
| 标题        | DynaPipe: Dynamic Layer Redistribution for Efficient Serving of LLMs with Pipeline Parallelism |
| 作者        | Hongxin Xu, Tianyu Guo, Xianwei Zhang                                                          |
| 发表场所 / 年份 | 文中未说明                                                                                          |
| 研究领域      | 大语言模型推理系统、流水线并行调度                                                                              |
| 关键词       | 流水线并行, LLM 推理, 负载均衡, KV 缓存迁移, 延迟预测                                                             |

### 资源链接

- 原始文件：本地上传文件（无外部链接）
- arXiv：文中未说明
- DOI：文中未说明
- 代码 / 数据集：<https://github.com/xhx1022/DynaPipe>

**标签（3-5 个）**：流水线并行 · LLM 推理服务 · 动态负载均衡 · KV 缓存迁移 · 延迟预测

DynaPipe 针对流水线并行（把模型层切成多段、分给不同 GPU 顺序执行的并行方式）推理中“末级阶段因采样操作而过载、拖累上游阶段空转”这一被忽视的瓶颈，提出运行时动态层重分配机制（执行时间预测器+气泡感知调度器+异步迁移协调器），在多种负载下将端到端时延降低 8%-41% [作者明确陈述，页码：1]。

## 2. 阅读范围与证据状态

本次分析覆盖 OCR 识别到的第 1-10 页正文及第 15 页附录，含摘要、方法设计（3 节）、实验（4 节）与相关工作（10 页）；图表证据覆盖 Figure 1-8 及附录一图，共 5 个批次 [作者明确陈述]。发表场所与年份 OCR 未能识别，写“文中未说明”；预测器拟合参数的具体拟合方法、 $\#mi("\Delta")$  触发阈值数值、误差百分比的计算公式、迁移开销的量化数值均未在 OCR 文本中出现，属论文本身缺失或本次分析无法确认的信息，非分析遗漏。以下判断中，论文明确陈述与实验支持的内容标注对应标签，涉及本文之外的通用知识（如 KV 缓存、张量并行原理）标注“背景知识”。

## 3. 根问题：论文究竟要解决什么

LLM 推理按解码步严格自回归依赖，下一个 token 必须等当前 token 生成后才能开始 [作者明确陈述，页码：2]。流水线并行（把模型很多层像工厂流水线一样分段，每段放一块 GPU，数据依次流过各段）让多块 GPU 并发工作，但各阶段之间存在顺序依赖 [作者明确陈述，页码：3]。问题在于：末级阶段除了要做与其他阶段相同的层前向计算，还要额外承担 token 采样（把模型算出的候选词概率转成实际输出词）这一步计算，这部分开销现有系统在分配层数时完全没有考虑 [作者明确陈述，页码：1-2]。这件事之所以重要，是因为自回归依赖会把末级过载反向传导：上游阶段必须等末级完成采样才能收到下一批输入，于是产生流水线气泡（某工位耗时突增导致其他工位空闲），直接拉低硬件利用率和端到端服务质量 [作者明确陈述，页码：2]。困难来源于采样开销不是固定值——它随请求组成（尤其是解码请求数量）动态变化，成功标准是要在这种动态性下仍能实时保持各阶段计算量对齐。**本文 story**：旧路线（均匀或静态层分配）之所以失效，是因为它们把层分配当作一次性决策，而采样与前向开销的比例本质上是随时间波动的随机量；本文的核心转折是引入运行时预测+动态调整，把“静态配置问题”改造成“持续感知-决策-迁移”的闭环。

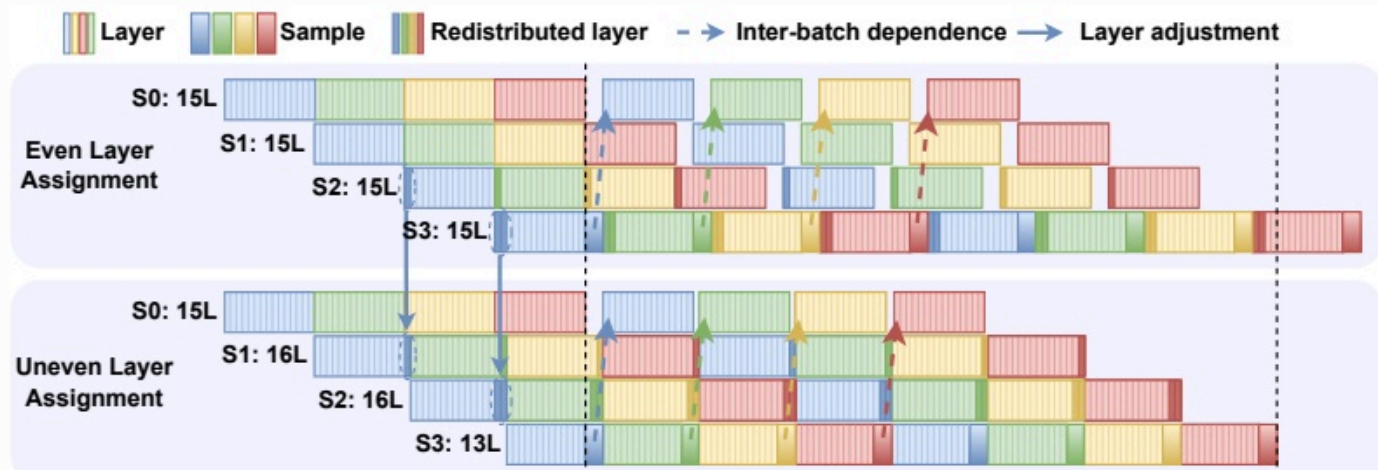


Figure 1: Illustration of pipeline bubbles during autoregressive decoding. Top: Conventional even layer allocation leads to idle upstream stages due to tail-stage sampling delays. Bottom: Uneven layer assignments to reduce idle time and improve hardware efficiency. "S0:15L" means stage 0 consists of 15 layers.

来源：OCR 第 2 页。

#### 图组解读

- **图组结论**：均匀分配下末级采样块显著拖长，上游阶梯状等待；不均匀分配后各阶段起止更对齐。
- **论证关系**：示意图直观支撑根问题机制，与 Figure 2 实测数据互补，为气泡感知调度器设计提供直觉铺垫。
- **适用范围**：层数（15/16/16/13）为示意性说明，非真实 profiling 结果，不能作为量化效果证据。

## 4. 旧方法为何不够

- **均匀层分配（传统 PP 系统）**：原本解决“如何把层公平切给多设备”的问题，前提是各阶段计算量相近；批判的具体限制是完全忽略采样开销，导致末级实际耗时高于其他阶段，产生结构性气泡 [作者明确陈述，页码：1-2]。本文用气泡感知调度器回应，Figure 1/2 与 4.2 节实验验证。
- **静态层重分配（Skywork-MoE、Megatron-LM 相关方法）**：原本试图通过预先减少末级层数补偿采样开销；其依赖前提是采样/前向比值近似恒定，但本文实测该比值近似正态分布（均值约 3.09 倍，页码：4），说明前提不成立，单一静态配置无法适应所有负载 [作者明确陈述，页码：3-4]。本文用动态调度器（ $\#mi("\Delta")$  公式实时判断是否重分配）回应，4.3 节 Figure 5 用不同 O:I 比率验证。
- **vLLM**：原本靠 PagedAttention 缓解显存碎片、用 PP 提升吞吐；批判点是固定尺寸 chunk 导致批间气泡，硬件利用率不佳 [作者明确陈述，页码：8]。DynaPipe 未声称解决此类批间气泡问题，二者比较仅在整体时延/SLO 层面（Figure 4），无直接对应模块。
- **SGLang**：原本靠结构化执行与 KV 复用提升效率，采用纯张量并行；批判点是高并发下跨设备通信开销大 [作者明确陈述，页码：8]。DynaPipe 基于 PP 架构，非 TP 优化，二者也仅整体比较（Figure 4）。
- **gLLM**：原本用自适应调度缓解批间气泡，是本文认定的 PP 架构下最优基线；批判点是忽视采样带来的非平凡开销，导致阶段间气泡限制端到端性能 [作者明确陈述，页码：3、8]。本文预测器+调度器+迁移协调器整体正是针对此限制设计，4.2 节 Figure 4、4.5 节 Figure 7 直接验证。
- **Vocabulary Parallelism（训练场景）**：原本为训练设计以缓解采样气泡；批判点是直接搬到推理需要更严格同步，显著增加系统复杂度，不适合服务部署 [作者明确陈述，页码：10]。此项为论证性陈述，无实验对比，“无法从当前 OCR 内容确认”是否有定量支持。

## 5. 核心洞见与假说

**核心洞见/贡献**：采样开销不是可忽略的末端步骤，而是一个随请求组成动态变化、平均约为单层前向耗时 3.09 倍的显著性能因子 [作者明确陈述，实验支持，页码：4]。因此解决末级过载不能靠一次性静态分配，必须运行时持续感知负载并动态调整——这是本文区别于旧路线的最小但关键的新想法。

**为落地该洞见所需的实现组件（非独立核心洞见本身）**：

1. 各阶段前向计算和末级采样时间可以被轻量模型实时、低成本地预测（每次预测仅耗时 0.5 微秒） [作者明确陈述，页码：10]；作者用离线 profiling 拟合线性模型来实现这一假设，前提是耗时与 token 数呈线性关系（由 Figure 2(a)(c) 支持）。
2. 层重分配可以在不阻塞流水线执行的前提下完成，依赖异步 KV cache 迁移与计算-通信重叠 [作者明确陈述，页码：2、6]；此前提在跨节点通信更差场景下可能打折扣（4.5 节收窄但仍有效）。
3. 稳定性窗口机制能过滤短期波动，避免频繁调整 [作者明确陈述，页码：6]；前提是波动确实是短期的，若负载持续大幅漂移，固定窗口大小的适用性作者未讨论。



## 6. 方法：从洞见到可执行系统

总览：系统输入每个 chunk 内各请求的 token 数  $n$ 、序列长度  $L$ 、请求数  $N$ 、解码请求数  $N_{\text{decode}}$ ，依次经过执行时间预测器、气泡感知调度器、迁移协调器三个模块，输出新的层-stage 映射并在不中断推理的情况下上线，最终减少流水线气泡、降低端到端时延 [作者明确陈述，页码：5]。

### 组件一：执行时间预测器

- **动机**：层重分配决策需要准确、实时的延迟估计，人工/静态估计跟不上采样与前向计算的动态变化 [作者明确陈述，页码：4]。
- **运行方式**：两个独立拟合模型——层前向时延  $T_{\text{layer}} = \sum_{i=1}^N (\phi_1 n_i + \phi_2 n_i L_i + \epsilon)$ ；采样时延  $T_{\text{sample}} = \alpha \cdot N_{\text{decode}} + \beta$  (页码：5)，系数由离线 profiling 拟合。
- **预期优势**：轻量 (0.5 微秒/次)，预测精度高 (层误差 4.95%，采样误差 0.31%) [作者明确陈述，实验支持，页码：9-10]。
- **与后续模块连接**：预测结果直接作为调度器计算  $\Delta$  的输入。**缺口**：拟合方法与数据规模“文中未说明”。

### 组件二：气泡感知调度器

- **动机**：仅有延迟预测不够，还需判断是否重分配、重分配多少层，以对齐各阶段计算时间 [作者明确陈述，页码：6]。
- **运行方式**：定义从未级移出的层数  $k$ 、接收方上游阶段数  $m = \min(k, \text{num}\{\text{stages}\} - 1)$ ，失衡指标  $\Delta = |T_{\text{sample}} - k \cdot T_{\text{layer}} - \frac{k}{m} \cdot T_{\text{layer}}|$  (页码：6)；引入稳定性窗口 (大小 25)，仅当某配置在窗口内持续一致出现才激活，过滤短期波动。
- **预期优势**：理想情况下  $\Delta \approx 0$  且  $m = \text{num}\{\text{stages}\} - 1$  时各阶段时间对齐 [作者明确陈述，页码：6]；窗口 = 25 时仅需 4 次调整即可接近最佳静态策略时延 [实验支持，页码：9]。**缺口**： $\Delta$  具体触发阈值数值“文中未说明”。

### 组件三：迁移协调器

- **动机**：层重分配若阻塞流水线执行会抵消收益，需要不打断在线推理的 KV cache 迁移方式 [作者明确陈述，页码：6]。
- **运行方式**：初始化阶段预加载潜在被重分配层的权重；触发迁移时比较新旧配置，源阶段异步发送、目标阶段异步接收 KV cache 并继续计算未受影响的层，仅在到达新分配层时才等待缓存 (通常此时已提前到达) [作者明确陈述，页码：6]。
- **预期优势**：利用流水线本身的并行性使计算与通信重叠，避免长时间停滞 [作者明确陈述，页码：6]。**缺口**：迁移本身的量化耗时占比“无法从当前 OCR 内容确认”。

前排论文大图 方法流水线 呼应：第 6 节：预测器-调度器-迁移协调器三模块协同运作

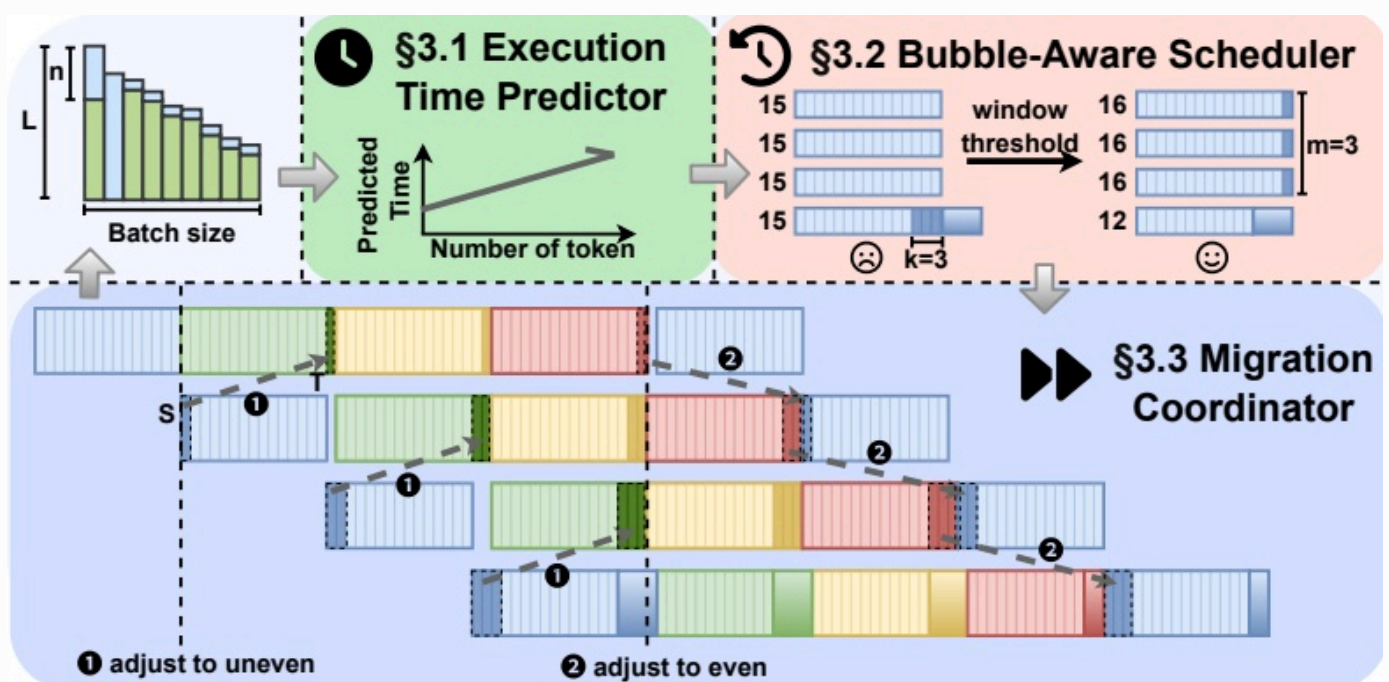


Figure 3: Overall architecture and workflow of DynaPipe. "S/T" means source/target stage.

来源：OCR 第 5 页。

### 图组解读

- **图组结论**：预测时间随 token 数上升；层配置从等长 ( $k=3$ ) 变为不等长 ( $m=3$ )；迁移时间轴显示源/目标阶段异步调整。
- **论证关系**：图解三大模块串联流程，与文字方法描述互补，不提供数值实验结果。
- **适用范围**：图中具体数字 (15/16/12、 $k=3$ 、 $m=3$ ) 是否取自真实实验运行无法从当前 OCR 内容确认。

## 7. 为什么它可能有效

- 预测器基于 Figure 2(a)(c) 展示的采样耗时/前向耗时与 token 数近似线性的实测关系建模，线性关系是拟合这类线性模型的前提 [作者明确陈述，页码：4]；若实际关系存在非线性区间 (如 Figure 2(c) 中 150–400 token 区间的分层聚集现象)，预测精度可能受影响，但该现象论文未讨论其成因 [基于文中逻辑的推断]。

- 调度器的  $\#mi("\Delta")$  越接近 0，各阶段计算时间越对齐，气泡越小；这一因果关系依赖预测器的准确性作为前提条件 [基于文中逻辑的推断]，一旦预测误差增大，调度决策的对齐效果可能失真。
- 迁移协调器通过异步传输与计算重叠来“隐藏”迁移开销，这一机制在通信条件良好（同节点 NVLink/PCIe）时更有效；在跨节点、仅走 TCP 的场景中，通信延迟增大会部分抵消重叠收益，这是 4.5 节收益收窄的可能机制解释 [基于文中逻辑的推断，页码：10]。

8. 实验与指标：证据是否支撑主张

8.1 实验问题与判定标准

- 效果问题：DynaPipe 相较 vLLM/gLLM/SGLang 是否降低端到端时延、提升 SLO 达标率——对应 4.2 节 Figure 4。
- 必要性问题：静态层重分配在动态负载下是否失效——对应 4.3 节 Figure 5（不同 O:I 比率）。
- 鲁棒性/机制折中问题：窗口阈值大小是否影响调整频率与时延——对应 4.4 节 Figure 6（消融）。
- 适用范围问题：跨节点通信更差场景、以及非主实验模型架构下方法是否仍有效——对应 4.5 节 Figure 7、附录 Figure（页 15）。
- 机制验证问题：预测器精度是否足够支撑调度决策——对应 4.6 节 Figure 8。

8.2 数据、任务、对比与运行设置

- 数据集：ShareGPT、Azure-Conv（页码：6-7）；4.3 节另用固定输入长度 512 的合成数据集构造不同 O:I 比率（页码：8）。
- 模型：Qwen2.5-14B/32B（主实验）；附录另测 Qwen3-30B-A3B（MoE）、Meta-Llama-3-8B-Instruct（Dense）（页码：6-7、15）。
- 硬件：4×A100-PCIe-40GB（页码：6-7）；4.5 节额外禁用 SHM/P2P/IB、强制走 TCP 模拟跨节点通信（页码：10）。
- 基线：vLLM(v0.8.5 V1)、gLLM、SGLang(v0.4.3.post2)（页码：6-7）；附录仅与 gLLM 对比。
- 其他设置：chunk size 2048，DynaPipe 窗口阈值 25（页码：7）。重复次数/统计方式：文中未说明。

8.3 指标字典与对应公式

- 指标与方向**： $\#mi("T_{\text{layer}}")$ ，单个 Transformer 层在一个 chunk 内的总执行延迟，越小越好。**定义与公式**：来源层级 **原文公式**； $\#mi("T_{\text{layer}}") = \sum_{i=1}^N (\phi_1 n_i + \phi_2 n_i L_i + \epsilon)$ ，页 5 正文显示公式（无 Eq. 编号）； $\#mi("N")$  为 chunk 内请求数， $\#mi("n_i")$ 、 $\#mi("L_i")$  为请求  $\#mi("i")$  的待算 token 数与序列长度， $\#mi("\phi_1, \phi_2, \epsilon")$  为 profiling 拟合参数。**实验用途**：供执行时间预测器建模层延迟，是调度器  $\#mi("\Delta")$  计算的输入。**读数与边界**：论文报告该模型预测的层执行时间平均相对误差为 4.95%（页码：9-10）；单位未注明，误差计算式本身“文中未说明”。
- 指标与方向**： $\#mi("T_{\text{sample}}")$ ，chunk 内采样操作总延迟，越小越好。**定义与公式**：来源层级 **原文公式**； $\#mi("T_{\text{sample}}") = \alpha \cdot N_{\text{decode}} + \beta$ ，页 5 显示公式（无编号）； $\#mi("N_{\text{decode}}")$  为 chunk 内解码请求数， $\#mi("\alpha, \beta")$  为拟合系数。**实验用途**：建模采样延迟，供调度器判断失衡。**读数与边界**：采样预测相对误差 0.31%（页码：9-10）；计算式“文中未说明”。
- 指标与方向**： $\#mi("\Delta")$ ，末级与上游重分配后计算时间差异，越接近 0 越好。**定义与公式**：来源层级 **原文公式**； $\#mi("\Delta") = \left| T_{\text{sample}} - k \cdot T_{\text{layer}} - \frac{k}{m} \cdot T_{\text{layer}} \right|$ ， $\#mi("m = \min(k, \text{num}_{\text{stages}} - 1))$ ，页 6 显示公式。**实验用途**：调度器判断是否达到阶段对齐、是否触发重分配。**读数与边界**：具体触发阈值数值“文中未说明”。
- 指标与方向**：E2EL（端到端时延），单请求从提交到完成的总耗时，越小越好。**定义与公式**：来源层级 **无可核验公式**；作者文字定义“total time from when a request is issued to when the response generation is completed”（页码：7），未给出聚合方式（均值/中位数）。**实验用途**：4.2/4.3/4.4 节所有时延对比、“降低 8%-41%”这一效果主张的核心指标。**读数与边界**：ShareGPT 上最高降低约 40%，Azure-Conv 上最高降低约 34%（页码：8）；聚合口径不明，无法核验分母。
- 指标与方向**：TTFT/TPOT，首 token 时延（秒）与平均单 token 生成时延（毫秒），越小越好。**定义与公式**：来源层级 **无可核验公式**，均为作者文字定义（页码：7），无计算式。**实验用途**：用于判定 SLO 达标（双阈值约束）。**读数与边界**：具体阈值见 Table 2，本次 OCR 未展开该表数值。
- 指标与方向**：SLO 达标率，满足 TTFT 与 TPOT 双阈值的请求占比，越大越好。**定义与公式**：来源层级 **无可核验公式**，文字定义（页码：7）。**实验用途**：4.2 节支撑“支持高 19% 请求速率”这一主张。**读数与边界**：DynaPipe 相比 gLLM 在 90% SLO 约束下支持请求速率高出 19%（页码：8）。
- 指标与方向**：采样/前向时延比，无量纲比值，用于说明采样开销量级。**定义与公式**：来源层级 **无可核验公式**；仅报告统计均值 3.09（页码：4），未给出计算式或分布参数。**实验用途**：论证“采样开销非平凡”这一核心动机（对应 Figure 2(b)）。**读数与边界**：分布近似正态，均值约 3.09 倍；具体方差/标准差“文中未说明”。

8.4 主结果、消融与证据边界

比较工作与被批判限制的呼应：

| 比较对象/基线 | 核心限制或失效条件                        | 本文对应模块          | 直接实验与仍未验证的边界                                  |
|---------|----------------------------------|-----------------|-----------------------------------------------|
| vLLM    | 固定尺寸 chunk 导致批间气泡 [作者明确陈述，页码：8]  | 无直接对应（问题类型不同）   | Figure 4 整体对比；未单独验证批间气泡机制本身                   |
| SGLang  | 高并发下 TP 通信开销大 [作者明确陈述，页码：8]      | 无直接对应（TP vs PP） | Figure 4 整体对比                                 |
| gLLM    | 忽视采样开销导致阶段间气泡 [作者明确陈述，页码：3、8]    | 预测器+调度器+迁移协调器整体 | Figure 4（4.2 节）、Figure 7（4.5 节多节点），19% 请求速率提升 |
| 静态层重分配  | 无法应对采样/前向比值的动态波动 [作者明确陈述，页码：3-4] | 气泡感知调度器         | Figure 5（4.3 节不同 O:I 比率），DynaPipe 全程优于静态策略    |

主张与证据强度：

| 主张                            | 直接证据（实验/图表）     | 证据强度与边界                                                      |
|-------------------------------|-----------------|--------------------------------------------------------------|
| E2EL 降低 8%-41%                | Figure 4（4.2 节） | [实验支持]；摘要"8%"下限对应场景"文中未说明"                                   |
| 支持高 19% 请求速率（vs gLLM，90% SLO） | Figure 4        | [实验支持]；仅限该 SLO 阈值设置                                          |
| 静态方案在动态负载下失效                  | Figure 5（4.3 节） | [实验支持]；限于固定输入长度 512 的合成数据集                                   |
| 窗口=25 为最优折中                   | Figure 6（4.4 节） | [实验支持]；仅在 32B 模型+ShareGPT 上验证，14B/Azure-Conv"无法从当前 OCR 内容确认" |
| 预测器精度高、开销可忽略                  | Figure 8（4.6 节） | [实验支持]；误差计算口径"文中未说明"                                         |
| 跨节点场景仍有效                      | Figure 7（4.5 节） | [实验支持]；收窄具体幅度"文中未说明"                                         |

- 各图表覆盖场景有限：窗口阈值实验仅在单一模型+单一数据集验证，未覆盖全部模型/数据集组合。
- 跨节点实验仅模拟 TCP-only 通信，未测试真实多机网络环境下的表现。
- 附录模型泛化实验仅与 gLLM 一个基线比较，不能推广到 vLLM/SGLang 的相对表现。
- 迁移协调器本身的量化开销（如迁移耗时占比）未见专门实验报告。

## 9. 图表解读与论证关系

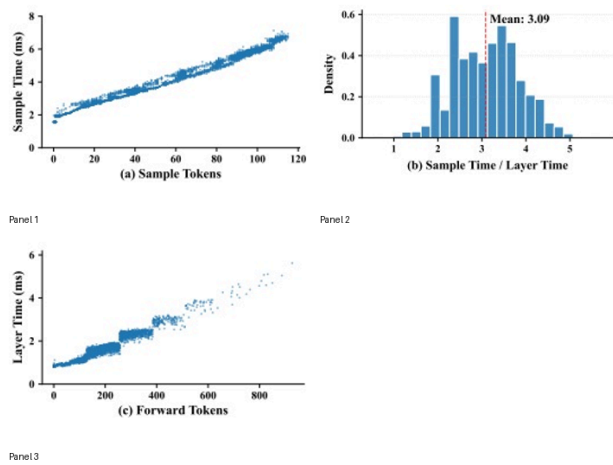


Figure 2: The overhead analysis of sampling and forwarding.

来源：OCR 第 4 页。

**图组解读 实验结论** 呼应：第 5 节：采样开销均值约为前向耗时 3.09 倍，随负载动态变化

- 图组结论**：采样耗时与前向耗时均随 token 数近似线性增长；两者比值近似正态分布，均值 3.09。
- 论证关系**：为预测器线性建模提供实证依据，证明"比值恒定"这一静态方案前提不成立，推动动态调度设计。
- 适用范围**：未说明 profiling 具体基于哪个模型或数据集；150-400 token 区间聚集现象成因未讨论。

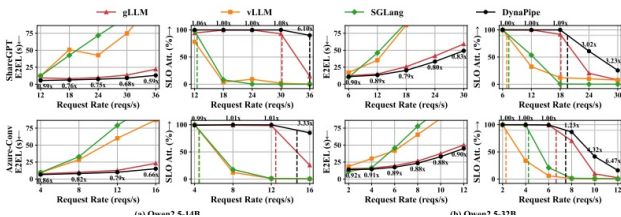


Figure 4: The average latency and SLO attainment rate of different LLM serving systems. DynaPipe achieves the lowest average latency and the highest SLO compliance across all workloads.

来源：OCR 第 7 页。

**图组解读 实验结论** 呼应：第 8 节：E2EL 降低 8%-41%，支持高 19% 请求速率（vs gLLM，90% SLO）

- 图组结论**：DynaPipe 在 E2EL 上全面低于 vLLM/gLLM/SGLang，SLO 达标率维持高位区间最长。
- 论证关系**：直接支撑两条效果主张，与 Figure 5/6/7 分别从必要性、鲁棒性、适用范围角度互补。
- 适用范围**：曲线相对位置不能单独验证通信开销、批间/阶段间气泡等内部机制；倍数标签定义文中未说明。

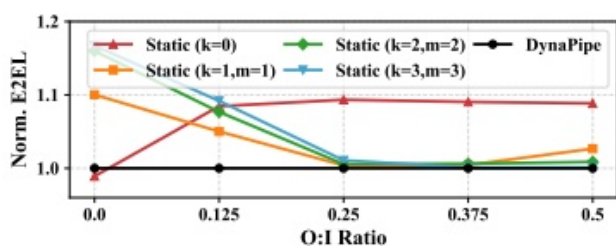


Figure 5: Normalized E2EL under different O:I ratios with various adjustment strategies.

来源：OCR 第 8 页。

**图组解读 对比基线** 呼应：第 4 节：静态层重分配无法应对采样/前向比值的动态波动

- 图组结论**：O:I=0 时静态 k=0 已接近 DynaPipe；O:I 增大后各静态策略峰值波动，DynaPipe 全程平稳偏低。
- 论证关系**：支撑"静态方案在动态负载下失效、动态方案更优"这一必要性主张，划定收益收窄的适用边界。
- 适用范围**：仅覆盖固定输入长度 512 的合成数据集，未说明模型规模或 GPU 配置。



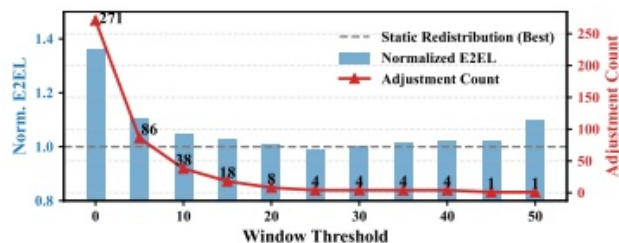


Figure 6: The effect of adjustment window threshold on E2EL and adjustment frequency.

来源：OCR 第 8 页。

**图组解读 消融验证** 呼应：第 6 节：稳定性窗口机制过滤短期波动，窗口=25 为折中最优

- **图组结论**：窗口=0 时调整次数最高 (271)、时延最高；窗口增大后调整次数趋稳，时延在 20-30 附近接近静态最佳基准。
- **论证关系**：验证窗口机制在调整开销与响应及时性间取得平衡，支撑窗口=25 折中最优的具体结论。
- **适用范围**：仅在 32B 模型+ShareGPT 数据集验证；窗口=25 对应柱高图中未单独标注。

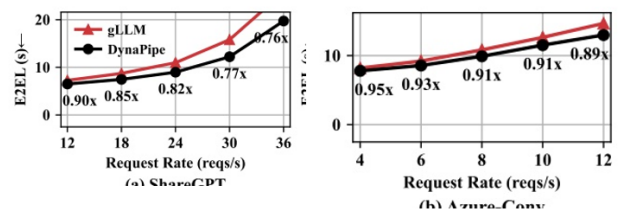


Figure 7: The latency comparison between DynaPipe and gLLM on Qwen2.5-14B (4 nodes).

来源：OCR 第 9 页。

**图组解读 适用边界** 呼应：第 8 节：跨节点通信更差场景下方法是否仍有效

- **图组结论**：ShareGPT 与 Azure-Conv 中 DynaPipe 在所有请求速率下均低于或接近 gLLM，比值均小于 1。
- **论证关系**：支撑“跨节点场景下 DynaPipe 仍优于 gLLM”这一适用范围主张；未含单节点对照曲线，收窄幅度无法仅凭本图核验。
- **适用范围**：“优化幅度较单节点收窄”的具体数值文中未说明。

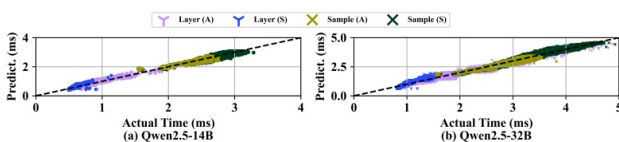


Figure 8: Actual vs. predicted execution time distribution. "A/S" means Azure-Conv/ShareGPT.

来源：OCR 第 9 页。

**图组解读 消融验证** 呼应：第 8 节：预测器精度是否足够支撑调度决策 (层误差 4.95%，采样误差 0.31%)

- **图组结论**：两个模型规模下，层与采样预测点均紧密聚集在  $y=x$  对角线附近，无系统性偏离。
- **论证关系**：支撑“预测器精度高”这一机制主张，为调度器依赖预测输入的前提假设提供验证。
- **适用范围**：图中无法逐点读出具体误差数值，误差计算口径文中未说明。

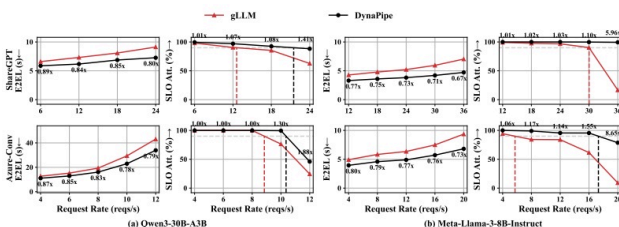


Figure 9: Performance comparison on MoE and Dense LLM models. DynaPipe consistently outperforms the baseline in both average latency and SLO compliance across all workloads.

来源：OCR 第 15 页。

**图组解读 适用边界** 呼应：第 8 节：非主实验模型架构下方法是否仍有效 (附录泛化实验)

- **图组结论**：Qwen3-30B-A3B 与 Llama-3-8B 上，DynaPipe 相较 gLLM 时延比例 0.67x-0.89x，SLO 达标率维持更久。
- **论证关系**：支撑“采样导致的阶段间不均衡是普遍问题、方法具有泛化性”这一适用边界主张。
- **适用范围**：仅与 gLLM 一个基线比较，不能推广到 vLLM/SGLang 在这些模型上的相对表现。

**表格证据：Table 1: Input / Output length.** **实验结论** 呼应：第 8 节：ShareGPT/Azure-Conv 数据集设置

- **图组结论**：表格记录两数据集请求的输入/输出长度统计，为 4.2 节负载设置提供依据。
- **论证关系**：与 Figure4 的 E2EL/SLO 实验共同定义实验负载特征，支撑效果主张的场景边界。
- **适用范围**：具体数值本次 OCR 未展开，无法逐项核验。

| Dataset    |     | #Input | #Output |
|------------|-----|--------|---------|
| ShareGPT   | P50 | 221    | 157     |
|            | P90 | 627    | 382     |
| Azure-Conv | P50 | 514    | 192     |
|            | P90 | 1008   | 412     |



**表格证据：**Table 2: SLO requirements. **实验结论** 呼应：第 8 节：TTFT 与 TPOT 双阈值构成 SLO 达标判定标准

- **图组结论：**表格给出 SLO 达标所需的 TTFT、TPOT 具体阈值，用于判定 4.2 节 SLO 达标率。
- **论证关系：**为 Figure 4 中 SLO Att. 子图的达标率计算提供阈值依据，支撑“支持高 19% 请求速率”主张。
- **适用范围：**具体阈值数值本次 OCR 未展开，无法逐项核验。

| Dataset    | Model | TTFT (s) | TPOT (ms) |
|------------|-------|----------|-----------|
| ShareGPT   | 14B   | 1        | 100       |
|            | 32B   | 4        | 250       |
| Azure-Conv | 14B   | 1        | 100       |
|            | 32B   | 4        | 150       |

## 10. 完整因果推理树

- 根问题：末级采样操作使流水线阶段计算不均衡，导致上游阶段空转、气泡增大、端到端时延升高
  - 旧方法限制：均匀/静态层分配未考虑动态变化的采样开销，前提（比值恒定）与实测不符 [作者明确陈述，页码：3-4]
  - 核心洞见/假设：采样开销显著且动态（均值约前向耗时 3.09 倍，近似正态分布），需运行时预测+动态调整
    - 方法模块：执行时间预测器（预测  $\#mi("T_{\{layer\}}")$ 、 $\#mi("T_{\{sample\}}")$ ）
      - 可验证预测：预测精度足够高、开销可忽略
        - 指标与实验：层/采样预测相对误差（4.95%/0.31%）、预测耗时 0.5 $\mu$ s
        - 图表证据：Figure 8 散点紧贴  $y=x$  线
          - 结论与适用边界：预测机制可靠，但误差计算口径“文中未说明”
      - 方法模块：气泡感知调度器（计算  $\#mi("\Delta")$ 、决定  $\#mi("k")$ 、 $\#mi("m")$ ）
        - 可验证预测：动态调度优于静态方案，窗口阈值可平衡开销与响应速度
          - 指标与实验：Norm. E2EL、调整次数、不同 O:I 比率下的时延
          - 图表证据：Figure 5（O:I 对比）、Figure 6（窗口消融）
            - 结论与适用边界：窗口=25 折中最优，但仅在 32B+ShareGPT 验证；O:I=0 时收益收窄
        - 方法模块：异步 KV cache 迁移协调器
          - 可验证预测：层重分配可非阻塞完成，跨节点场景仍有效
            - 指标与实验：4.5 节跨节点 E2EL 对比
            - 图表证据：Figure 7
              - 结论与适用边界：跨节点仍优于 gLLM，但收窄幅度未量化，迁移自身开销未单独测量（未被当前证据直接验证）
    - 整体系统效果
      - 指标与实验：E2EL、SLO 达标率、请求速率
      - 图表证据：Figure 4（主实验）、附录图（其他模型架构）
        - 结论与适用边界：8%-41% 时延降低、19% 请求速率提升，在 4 $\times$ A100-PCIe 单节点+两数据集+四种模型架构下验证；未覆盖训练场景、未与 vLLM/SGLang 在附录模型上比较

## 11. 结论、贡献与边界

**贡献总结：**论文明确提出的贡献包括（1）识别并量化 PP 推理中被忽视的末级采样气泡瓶颈；（2）DynaPipe 动态层重分配方案（预测器+调度器）；（3）异步 KV cache 迁移协调器 [作者明确陈述，页码：2]。

**证据充分的结论：**DynaPipe 在 ShareGPT/Azure-Conv、Qwen2.5-14B/32B、4 $\times$ A100 环境下相较 vLLM/gLLM/SGLang 降低端到端时延、提升 SLO 达标率 [实验支持，页码：8]；预测器精度高、开销可忽略 [实验支持，页码：9-10]；窗口机制能权衡调整开销与响应速度 [实验支持，页码：9]。

**尚属推断的意义：**预测精度依赖 token 数与耗时的线性关系，该关系在部分区间存在局部离散现象，是否影响极端负载下的调度决策 [基于文中逻辑的推断]；跨节点场景收益收窄的具体机制（通信延迟部分抵消重叠收益）为推断而非作者直接量化 [基于文中逻辑的推断]。

**适用条件：**方法收益与 O:I 比率相关，纯 prefill 场景（采样开销小）下收益与静态方案相近而非显著更优 [作者明确陈述，页码：8]；验证覆盖单节点及 TCP 模拟跨节点，未见真实多机网络环境。

**局限（作者自述，页码：10）：**运行时迁移仍有通信开销（局部小幅波动）；需预留额外显存存储待接收权重，降低显存利用效率；未来方向为压缩传输、offloading、集成 Vocabulary Parallelism。

**下一步验证：**迁移开销的独立量化测量、窗口阈值在其他模型/数据集组合下的普适性、与更多调度类工作（如 TD-Pipe、Llumnix）的直接实验对比，目前均“文中未说明”或“无法从当前 OCR 内容确认”。

**审稿式风险检查：**

- 贡献新意：核心洞见（采样开销动态、非平凡）有 Figure 2 实测数据支撑，风险较低；当前证据未显示该风险。
- 写作/可复现性：代码已公开（GitHub 链接），但预测器拟合参数的具体拟合方法未披露，复现该部分存在障碍 [基于文中逻辑的推断]。
- 效果大小：8%-41% 的区间跨度较大，摘要“8%”下限对应场景未明确指出，读者难以判断最保守收益出现在何种负载 [基于文中逻辑的推断]。

- 实验覆盖：窗口阈值消融仅在单一模型+单一数据集验证，未覆盖全部实验矩阵；跨节点实验为模拟而非真实多机环境 [作者明确陈述，页码：9-10]。
- 方法设计：迁移协调器自身引入的量化开销未被单独测量，其“非阻塞”效果目前仅有整体端到端时延的间接证据支撑 [基于文中逻辑的推断]。

## 12. 术语与第一性原理学习路径

### 12.1 核心术语

- **术语**：流水线并行 (Pipeline Parallelism) ——把模型层切分为多个顺序阶段，每阶段分配给不同设备并发执行的并行方式。**第一性原理角色**：受设备数与层数的资源约束，连接“单设备计算能力有限”与“多设备协同吞吐”这两个基本量 [背景知识]。**本文位置**：本文方法建立在 PP 架构之上，重新分配的对象即是各阶段承担的层数 [作者明确陈述，页码：3]。
- **术语**：流水线气泡 (Pipeline Bubble) ——因某阶段耗时突增导致其他阶段空闲等待的现象。**第一性原理角色**：由自回归依赖这一因果约束产生：下游必须等上游完成才能继续，任何一环延迟都会传导为空闲 [作者明确陈述，页码：2]。**本文位置**：本文的根本问题即末级采样导致的气泡，是全文优化目标 [作者明确陈述，页码：1-2]。
- **术语**：采样 (Sampling) ——模型算出候选词概率后按策略选出下一个生成词的操作。**第一性原理角色**：仅在生成阶段的每一步发生，耗时随解码请求数量线性增长 (Figure 2a) [作者明确陈述，页码：4]。**本文位置**：末级阶段独有的额外负载来源，是本文识别的核心瓶颈 [作者明确陈述，页码：1]。
- **术语**：KV Cache (键值缓存) ——缓存此前 token 计算出的键值向量以避免重复计算。**第一性原理角色**：受显存容量约束，连接“历史计算结果”与“当前生成步骤的输入”两个基本量 [背景知识]。**本文位置**：迁移协调器需要在层重分配时异步迁移对应 KV cache [作者明确陈述，页码：6]。
- **术语**：执行时间预测器 (Execution Time Predictor) ——基于离线拟合的线性模型，实时估计层前向与采样时延。**第一性原理角色**：以 token 数、序列长度、请求组成为输入变量，受“预测必须足够快、足够准”的约束，连接“负载特征”与“调度决策”两个环节 [作者明确陈述，页码：5]。**本文位置**：三大模块中的第一环，为调度器提供决策依据 [作者明确陈述，页码：5]。
- **术语**：气泡感知调度器 (Bubble-Aware Scheduler) ——依据失衡指标  $\#mi(\Delta)$  决定是否及如何重分配层的模块。**第一性原理角色**：受“重分配需带来净收益”约束，连接“预测的时延差异”与“具体层-阶段映射”两个量 [作者明确陈述，页码：6]。**本文位置**：三大模块中的第二环，输出新的层分配配置 [作者明确陈述，页码：6]。
- **术语**：异步 KV cache 迁移协调器 (Migration Coordinator) ——使层重分配在不阻塞流水线执行的前提下完成的机制。**第一性原理角色**：受“通信与计算需重叠才能隐藏开销”这一约束，连接“新的层分配决策”与“实际系统状态切换”两个环节 [作者明确陈述，页码：6]。**本文位置**：三大模块中的第三环，负责决策落地 [作者明确陈述，页码：6]。
- **术语**：SLO (服务级别目标) 与 TTFT/TPOT——响应速度承诺门槛；首 token 等待时间；后续每 token 平均间隔。**第一性原理角色**：受用户体验的时间敏感度约束，连接“系统内部计算延迟”与“用户可感知的服务质量”两个层面 [作者明确陈述，页码：7]。**本文位置**：作为实验中判定服务质量达标与否的核心指标 [作者明确陈述，页码：7]。

### 12.2 学习路径

1. **自回归生成**：LLM 逐词生成、下一步依赖上一步的结果，这是理解“为什么末级延迟会传导为上游空转”的最基本约束。
2. **流水线并行与阶段依赖**：理解多设备如何分工协作，才能理解“层数分配”这一决策变量的含义。
3. **采样操作的计算特性**：理解采样为何只发生在末级、耗时如何随请求数变化，才能理解本文识别的瓶颈来源。
4. **延迟预测与线性建模**：理解通过历史数据拟合简单模型估计耗时的思路，才能理解执行时间预测器的作用。
5. **KV cache 与异步通信**：理解缓存迁移和计算-通信重叠的基本机制，才能理解迁移协调器如何做到“重分配不阻塞”。